

- The Computer graphics is one of the most effective and commonly used way to ^{collect} information in form of graphics object such as pictures, charts, graphs and diagram instead of simple text.
- Computer graphics is a process of generating, manipulating, storing and displaying graphics object.

Ex: Such as pictures, charts, graphs, diagrams,

Basic Graphics System

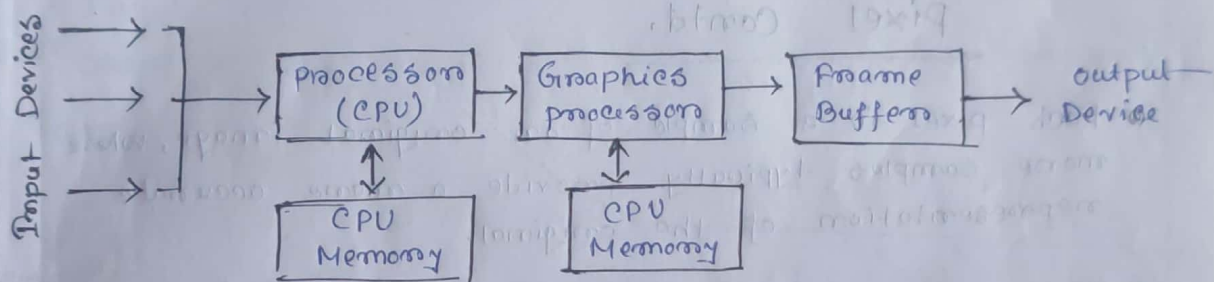
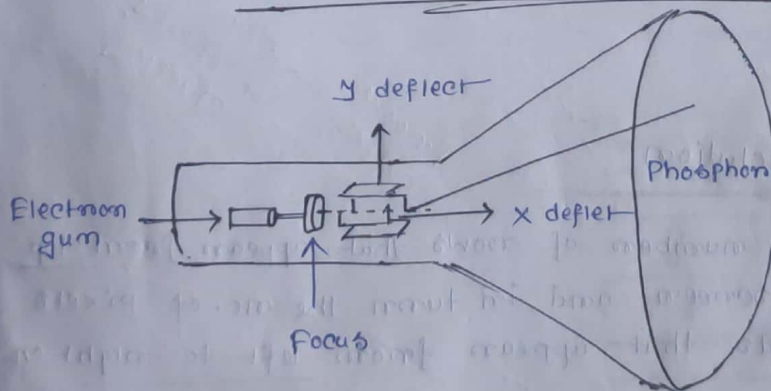


Image formed in frame buffer

CRT (Cathode Ray Tube)



A CRT is a specialized vacuum tube in which images are produced when an electron beam strikes a phosphorescent surface. Most desktop computer displays make use of CRT's. The CRT in a computer display is similar to the "picture tube" in a television receiver.

Can be used either as a line-drawing device (calligraphic) or to display contents of frame buffer (monitor mode)

Pixel

- Graphics should be generated by controlling the pixel.
- The control is achieved by setting the intensity and colour of the pixel which compose screen.
- The process of determining the appropriate pixels for representing picture on graphics object is known as "masterization".
- The process of representing continuous picture on graphics object as a collection of discrete pixels is called "Scan Conversion".

Pixel Contd.

- Each pixel is a sample of an original image, where more samples typically provide a more accurate representation of the original.
- The intensity of each pixel is variable; in color systems, each pixel has typically three or four components such as ~~red~~ red, green and blue, or cyan, magenta, yellow and black.

Resolution

- Resolution is the number of rows that appear from top to bottom of a screen and in turn the no. of pixels or pixel elements that appear from left to right on each scan line.
- Based on this resolution only the effect of picture appears on screen.
- In other words greater the resolution greater will be the clarity of picture. That is resolution value is directly proportional to clarity of picture.

Frame buffer → For intensity checking

Contd.

- Thus we ~~can~~ say that Computer graphics makes it possible to express data in pictorial form.

- In computer graphics objects are presented as a collection of discrete picture elements.

- Picture Element = pixel = pe

- The pixel is the smallest screen elements.

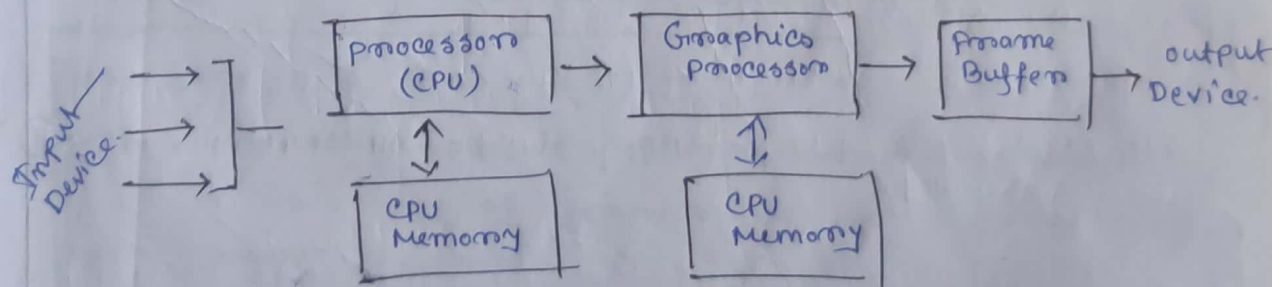
- pixels are normally arranged in a regular 2D grid and are often represented using dots and squares.

- Input Device.

Track ~~Stack~~ ball, Space-Ball, joystick, DD, light pens, Track pens.

✓ Frame Buffer : A framebuffer is a portion of random-access-memory (RAM) containing a bitmap that drives a video display.

It is a memory buffer containing data representing all the pixels in a complete video frame.



Picture Formed in a Frame Buffer

various point.

It is a large continuous piece of memory in which the intensity values for all pixel are least.

Application of Computer Graphics : ① Animation / computer arts :

CG methods are widely used in both fine art and commercial art application.

② Internet : There is a large amount of multimedia content available on internet.

③ Desktop publishing (DTP) : DTP allows the use of computer graphics for creation of business graphics, charts, maps, and other pictorial features. It is also related to journal, newsletter, books, printing etc.

④ Medical Application : Now a days, several complex surgeries are done by using robots. To observe continuously ^{the} operation process, the monitors need to be fixed with some special graphics hardware and software.

⑤ Computer vision : Camera tracking is an interesting application of computer vision to create visual effects for cinema and broadcast. It also has its application in medicine, military, security, quality inspection, robotics, and other things.

• Frame Buffer

Classification of Computer Graphics :

There are two types of CG —

① Non-interactive - passive - offline Computer graphics :

In this type of graphics the user has no control over graphics / picture.

Ex: A static website, TV monitor etc.

② Interactive - Active - online Computer graphics :

In this type of graphics the user has control over graphics.

Ex: Dynamic website, cartoon, video games etc.

Other category of Computer graphics are —

business and presentation graphics, scientific graphics, scale drawing, cartoon & artwork like advertisement, GUI (Graphical user Interface)

Characteristics of CG:

- ① It is interactive in nature.
- ② It is very user friendly.
- ③ It ^{should} provides a tool called as motion dynamics which allows users to move objects.
- ④ It should provides update dynamics so that users can manipulate the object like change the shape, colour and other properties.
- ⑤ It has the ability to show moving picture that is animation.

Resolution: There are two types of resolution —

- ① Image Resolution: It is defined as the distance from one pixel to the next pixel. That is pixel spacing.
- ② Screen Resolution: It is defined as the no. of pixels in the horizontal and vertical directions on the screen.

[N:B] Increasing resolution means making image smaller because the screen would be displaying more pixels per inch.

for example a resolution of (1024×640) means —
1024 pixels are going horizontally and 640 pixels are going vertically.

Bit Depth: It is defined as the no. of bits assigned to each pixel in the image. It specifies the no. of colours that a monitor can display.

Ex: one byte (8 bit) per pixel represents —
 $2^8 = 256$ colours for each pixel.

Dot pitch: It is defined as the measurement of the diagonal distance between two ~~light~~ like - colour pixels on a display screen. (RGB)

It is measured in millimetre.

Dot pitch, Bit Depth and resolution are the three most important qualities parameters for the measuring quality of display image.

Line Drawing Algorithm (DDA) : ✓ chapter-3-

Digital Differential Analyzer.

It is a scan conversion line algorithm based on either calculating Δy or Δx .

```
void lineDDA (int xa, int ya, int xb, int yb)
```

```
{ int dx = xb - xa, dy = yb - ya, steps, K;
```

```
float xinc, yinc, x = xa, y = ya;
```

```
if (abs(dx) > abs(dy)) steps = abs(dx);
```

```
else
```

```
steps = abs(dy);
```

```
xinc = dx / (float) steps
```

```
yinc = dy / (float) steps
```

```
setpixels (Round(x), Round(y));
```

```
for (K = 0; K < steps; K++) {
```

```
    x += xinc;
```

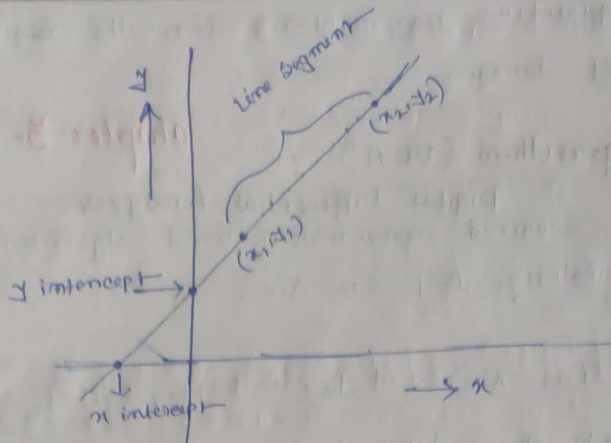
```
    y += yinc;
```

```
    setpixels (Round(x), Round(y)); }
```

The DDA algorithm (Digital Differential Analyzer) is a scan conversion line algorithm based on calculating either Δy and Δx .

— Line is a infinite in dimension.

Line segment have 2 different point (endpoints).



Line is infinite in dimension.

A line segment has two confine end point.

Line equation:

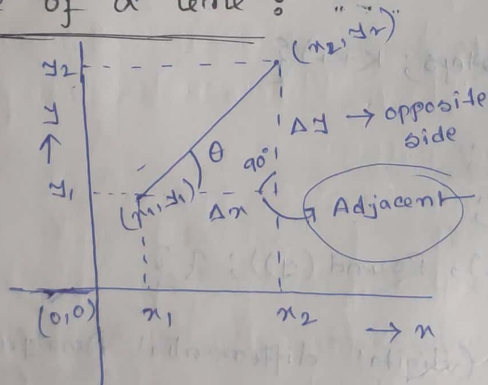
(i) $y = mx + c \rightarrow$ intercepting point on y axis.

Slope of a line

(ii) $\frac{x}{a} + \frac{y}{b} = 1$

(iii) $(y - y_1) = \frac{y_2 - y_1}{x_2 - x_1} (x - x_1)$

Slope of a line :

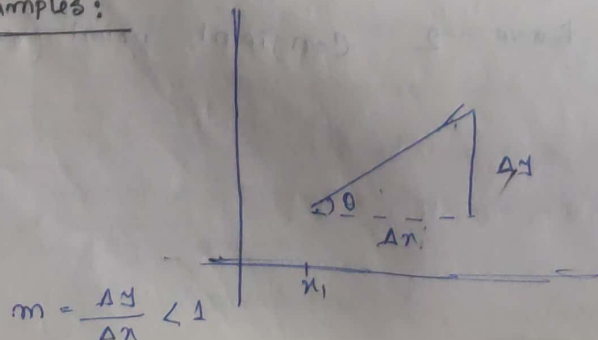


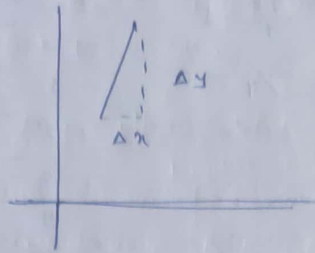
$\frac{dy}{dx} = \frac{\Delta y}{\Delta x}$

$m = \frac{\Delta y}{\Delta x}$

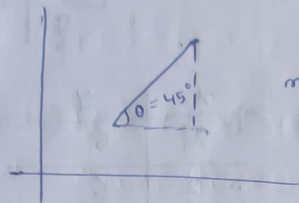
$= \tan \theta$

Examples:





$$m = \frac{\Delta y}{\Delta x} > 1$$



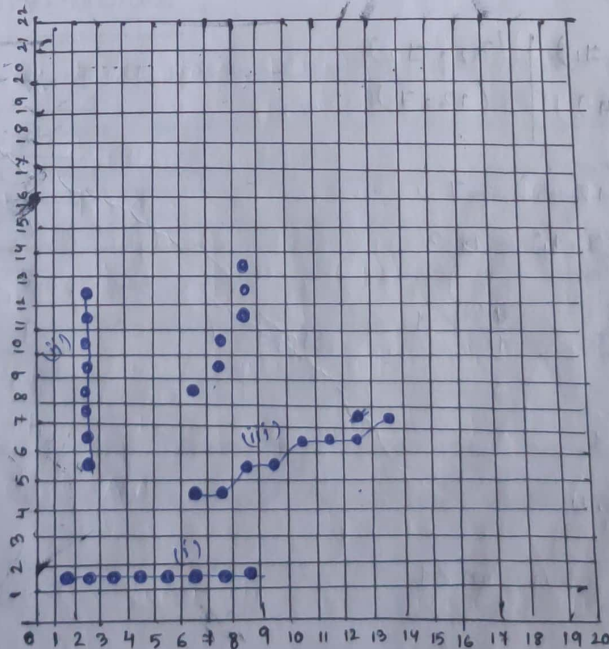
$$m = \frac{\Delta y}{\Delta x} = 1$$

So, lines which have angle $> 45^\circ$ have slope > 1

" $< 45^\circ$ " " " " < 1

" $= 45^\circ$ " " " " $= 1$

DD Algorithm :



Example - 1 $(x_1, y_1) \mid (x_2, y_2)$
 $(2, 2) \mid (9, 2)$

$$\Delta x = 9 - 2 = 7$$

$$\Delta y = 2 - 2 = 0$$

$$m = \frac{\Delta y}{\Delta x} = \frac{0}{7} = 0, \text{ steps} = 7$$

$$x_{inc} = \frac{\Delta x}{\text{steps}} = \frac{7}{7} = 1$$

$$y_{inc} = \frac{\Delta y}{\text{steps}} = \frac{0}{7} = 0$$

$$\therefore m = 0$$

	x	y
	2	2
1	3	2
2	4	2
3	5	2
4	6	2
5	7	2
6	8	2
7	9	2

$$(ii) \quad \begin{array}{l} (x_1, y_1) \\ (2, 5) \end{array} \quad \begin{array}{l} (x_2, y_2) \\ (2, 12) \end{array}$$

$$\Delta x = 2 - 2 = 0$$

$$\Delta y = 12 - 5 = 7$$

$$\text{steps} = 7$$

$$m = \frac{\Delta y}{\Delta x} = \frac{7}{0} = \infty > 1$$

$$x_{inc} = \frac{0}{7} = 0$$

$$y_{inc} = \frac{7}{7} = 1$$

$$\therefore m > 1$$

x	y
2	5
2	6
2	7
2	8
2	9
2	10
2	11
2	12

$$(iii) \quad \begin{array}{l} (x_1, y_1) \\ (5, 4) \end{array} \quad \begin{array}{l} (x_2, y_2) \\ (12, 7) \end{array}$$

$$\Delta x = (12 - 5) = 7$$

$$\Delta y = (7 - 4) = 3$$

$$m = \frac{\Delta y}{\Delta x} = \frac{3}{7} < 1$$

$$\text{steps} = 7$$

$$x_{inc} = 1$$

$$y_{inc} = 0.4$$

This line has $m < 1$

x	y	
5	4	4
6	4.4	4
7	4.8	5
8	5.2	5
9	5.6	6
10	6	6
11	6.4	6
12	6.8	7

$$\boxed{\begin{array}{l} x_{k+1} = x_k + 1 \\ y_{k+1} = y_k + m \end{array}}$$

If m of the line is less than 1, then x will increment by 1 and y by the slope (that is m).

Scan conversion is the process of representing continuous graphics objects as a collection of discrete pixels. Each pixel can have either on or off state.

(iv) $(x_1, y_1) \mid (x_2, y_2)$
 $(5, 4) \mid (10, 15)$

$$\Delta x = 5 \quad m = \frac{\Delta y}{\Delta x} = \frac{8}{5} > 1$$

$$\text{Steps} = 8$$

$$x_{\text{inc}} = \frac{5}{8} = 0.6$$

$$y_{\text{inc}} = \frac{8}{8} = 1$$

	x	y
5	5	4
6	5.6	8
7	6.2	9
8	6.8	10
9	7.4	11
10	8	12
11	8.6	13
12	9.2	14
13	9.8	15

(v) $(x_1, y_1) \mid (x_2, y_2)$
 $(12, 9) \mid (17, 14)$

$$m = 1$$

$$\Delta x = 5 \quad m = \frac{\Delta y}{\Delta x} = 1$$

$$\Delta y = 5$$

$$\text{Steps} = 5$$

$$x_{\text{inc}} = 5/5 = 1$$

$$y_{\text{inc}} = 5/5 = 1$$

	x	y
12	12	9
13	13	10
14	14	11
15	15	12
16	16	13
17	17	14

Disadvantage: We can't get exact ~~time~~ straight line in this case.

Scan Conversion:

The process of conversion of the rasterized picture stored in a frame buffer to the rigid display pattern of video is called scan conversion. Hence scan conversion is a continuous to discrete transformation.

Thus we need to convert the cartesian co-ordinates to the system understandable points.

We need follow certain states _____

Step 1: Identify the starting address corresponding to the line on which the point is to be displayed.

Step 2: Find the byte address at which the point is to be displayed.

Step 3: Compute the value ~~of~~ from the byte that represents the points.

Step 4: Logically or the calculated value with the present value of the byte.

Step 5: Store the value found in step 4.

Step 6: STOP

Vector generation: The process of turning ^{'ON'} the pixels from a line segment is called vector generation.

A scan conversion of line means locating pixel coordinates near an ideal straight line.

Limitations of DDA:

- (1) The rounding operation & floating point arithmetic are time consuming procedures.
- (2) Round-off error can cause the calculated pixel position to drift away from the true line path for long line segment.

The Bresenham Line drawing Algorithm:

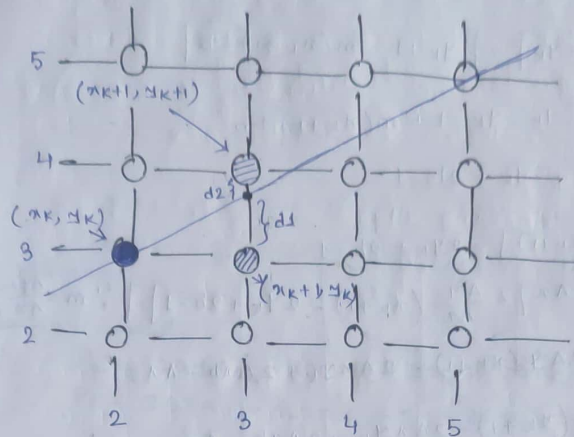
- The Bresenham algorithm is another incremental scan conversion algorithm.
- The big advantage of this algorithm is that it uses only integer calculations.

Principle: In this algorithm we try to find out optimal raster locations to represent straight lines. The x or y increment is set to 1 in x or y axis respectively depending upon the slope of line.

if $m \leq 1$ increment is set in x -axis else
the increment in other direction ^(x or y) is calculated by selecting the closest pixel from the true lines at each position of x or y based on the decision variable or error term.

(An error term is defined as the distance between the actual line location and the nearest pixel.)

Our aim is to select pixels that are closest to the true, expected line.



At sample position x_{k+1} the vertical separations from the mathematical line are labelled as d_1 and d_2 .

The y coordinate on the mathematical line at x_{k+1} is —

$$y = mx + b$$

$$\text{or, } y = m(x_{k+1}) + b$$

$$\therefore d_1 = \text{actual } y - y_k = y - y_k$$

$$= (m(x_{k+1}) + b) - y_k \quad [\because y = m(x_{k+1}) + b]$$

$$= m(x_{k+1}) + b - y_k$$

$$d_2 = y_{k+1} - y$$

$$= y_{k+1} - (m(x_{k+1}) + b)$$

$\therefore$ Def. Decision variable = if $d_1 - d_2 < 0$, (means d_1 is smaller), y_k

if $d_1 - d_2 > 0$ (means d_2 is smaller), y_{k+1}

$\therefore$ This both are decision variable.

Now, $d_1 - d_2$

$$= [m(x_{k+1}) + b - y_k] - [y_{k+1} - m(x_{k+1}) - b]$$

$$= m(x_{k+1}) + b - y_k - y_{k+1} + m(x_{k+1}) + b$$

$$= 2[m(x_{k+1})] - 2y_k + 2b - 1$$

$$\therefore d_1 - d_2 = 2[m(x_{k+1})] - 2y_k + 2b - 1$$

$$\text{or, } \Delta x (d_1 - d_2) = \Delta x [2 \frac{\Delta y}{\Delta x} (x_{k+1}) - 2y_k + 2b - 1] \quad \left[\because m = \frac{\Delta y}{\Delta x} \right]$$

$$\text{or, } \Delta x (d_1 - d_2) = 2\Delta y (x_{k+1}) - 2\Delta x y_k + 2\Delta x b - \Delta x$$

Decision
Parameter P_k

$$= 2\Delta y (x_{k+1}) - 2\Delta x y_k + 2\Delta x b - \Delta x$$

$$= 2\Delta y x_k - 2\Delta x y_k + 2\Delta y + 2\Delta x b - \Delta x$$

$x \rightarrow \text{constant value}$

$$\therefore P_k = 2\Delta y x_k - 2\Delta x y_k$$

Bresenham's Line Algorithm (cont):

- Input two end-points and store (x_0, y_0) in the frame buffer.
- Plot (x_0, y_0) to be the first point.
- Calculate the constants Δx , Δy , $2\Delta y$ and $2\Delta y - 2\Delta x$ and obtain the starting value for the decision parameter $a_0 p_0 = 2\Delta y - \Delta x$.
- At each x_k along the line starting at $k=0$, perform the following test.

If $P_k < 0$ plot (x_{k+1}, y_k) and $P_{k+1} = P_k + 2\Delta y$
otherwise,

$$\text{plot } (x_{k+1}, y_{k+1}) \text{ and } P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

- perform step 4 $\Delta x - 1$ times.

$$P_k = 2\Delta y x_k - 2\Delta x y_k$$

$$P_{k+1} \text{ or } P_{\text{next}} = 2\Delta y x_{\text{next}} - 2\Delta x y_{\text{next}}$$

$$P_{\text{next}} - P_k = [2\Delta y x_{\text{next}} - 2\Delta x y_{\text{next}} - 2\Delta y x_k + 2\Delta x y_k]$$

$$= 2\Delta y (x_{\text{next}} - x_k) - 2\Delta x (y_{\text{next}} - y_k)$$

if $P_{\text{next}} - P_k < 0$

$$P_{\text{next}} = P_k + 2\Delta y (x_{k+1} - x_k) - 2\Delta x (y_k - y_k)$$

$x_{\text{next}} = x_{k+1}$

Slope is a floating value

$$m = \frac{\Delta y}{\Delta x} - \Delta x$$

$$P_{next} = P_k + 2\Delta y (x_{k+1} - x_k)$$

where y_{next} as y_k

if $P_{next} - P_k > 0$

$$P_{next} = P_k + 2\Delta y - 2\Delta x$$

where y_{next} as y_{k+1}

Initial value,

$$P_k = 2\Delta y x_k - 2\Delta x y_k + 2\Delta y + 2\Delta x c - \Delta x$$

$$P_1 = 2\Delta y x_1 - 2\Delta x y_1 + 2\Delta y + 2\Delta x [y_1 - \frac{\Delta y}{\Delta x} x_1] - \Delta x$$

$$= 2\Delta y x_1 - 2\Delta x y_1 + 2\Delta y + 2\Delta x y_1 - 2\Delta x x_1 - \Delta x$$

$$P_1 = 2\Delta y - \Delta x$$

$$c = y_1 - \left(\frac{\Delta y}{\Delta x} x_1 \right)$$

$$y_1 = m x_1 + c$$

$$y = \frac{\Delta y}{\Delta x} x + c$$

$$c = y_1 - \frac{\Delta y}{\Delta x} x_1$$

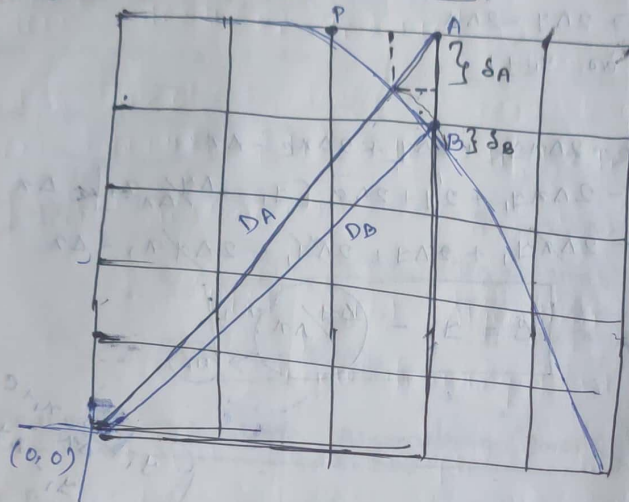
Circle Generating Algorithm:

A circle is defined as the set of points that are at a given distance 'r' from a centre position x_c and y_c . This distance relationship is expressed in —

(~~x~~ x_c cartesian coordinates are —

$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

Bresenham circle drawing Algo:



In Bresenham circle drawing Algorithm the integer operation involving square of the pixel separation distances. Here we perform $90^\circ \rightarrow 45^\circ$ calculation, where x

moves in $+x$ direction and y moves in $-y$ direction. Distance of pixel 'A' from origin $(0,0)$ —

$$DA = \sqrt{(x_i+1)^2 + (y_i)^2}$$

$$DB = \sqrt{(x_i+1)^2 + (y_i-1)^2} \quad \left[\text{Distance of pixel 'B' from origin } (0,0) \right]$$

The distance of pixel A and B from the true circle are —

$$\delta_A = DA - r \quad \text{and} \quad \delta_B = DB - r$$

To simplify the decision variable eqⁿ we avoid the square root.

$$\delta_A = DA^2 - r^2$$

$$\delta_A = ((x_i+1)^2 + y_i^2) - r^2$$

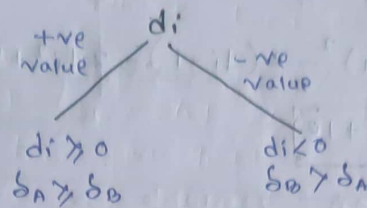
$$\delta_B = ((x_i+1)^2 + (y_i-1)^2) - r^2$$

$$\begin{aligned} x_{i+1} &= x_i + 1 \\ \downarrow & \quad \left[\begin{array}{l} \text{current} \\ \text{pixel} \\ x_i \text{ and} \\ y_i \end{array} \right] \\ \text{Next pixel} & \quad \left[\begin{array}{l} \text{of } x_i \\ y_i \end{array} \right] \\ y_{i+1} &= y_i - 1 \end{aligned}$$

$\downarrow$
Next pixel
of y_i

We can observe that δ_A is always +ve and δ_B is always -ve. So, we can define decision variable d_i as,

$$d_i = \delta_A + \delta_B$$
 depending on this eqn we can choose pixel A or B which is nearer to the circle boundary.



Case 1: $x_{i+1} = x_i + 1$

$d_i > 0, y_{i+1} = y_i - 1$

Case 2: $x_{i+1} = x_i + 1$

$y_{i+1} = y_i$
(No change)

Now, the eqn of d_i at starting point i.e. $x=0, y=m$ at $(0, m)$

$$\begin{aligned}
 d_i &= \delta_A + \delta_B \\
 &= (x_{i+1})^2 + y_i^2 - m^2 + (x_{i+1})^2 + (y_i - 1)^2 - m^2 \\
 &= (0+1)^2 + m^2 - m^2 + (0+1)^2 + (m-1)^2 - m^2 \\
 &= 1 + 1 + m^2 + 1 - 2m - m^2
 \end{aligned}$$

$$\therefore d_i = 3 - 2m \quad \text{at position } (0, m)$$

Now, the another eqn of next decision parameter d_{i+1} is equal to

$$\begin{aligned}
 d_i &= \delta_A + \delta_B \\
 &= (x_{i+1})^2 + y_i^2 - m^2 + (x_{i+1})^2 + (y_i - 1)^2 - m^2 \\
 &= 2(x_{i+1})^2 + y_i^2 - 2m^2 + y_i^2 - 2y_i + 1
 \end{aligned}$$

$$d_i = 2(x_{i+1})^2 + 2y_i^2 - 2m^2 - 2y_i + 1$$

$$\begin{aligned}
 \therefore d_{i+1} &= 2(x_{i+1} + 1)^2 + 2(y_{i+1})^2 - 2m^2 + 1 \\
 &= 2(x_{i+2})^2 + 2(y_{i+1})^2 - 2y_{i+1} - 2m^2 + 1
 \end{aligned}$$

Now calculate:

$$\begin{aligned}
 d_{i+1} - d_i &= 2(x_{i+2})^2 + 2(y_{i+1})^2 - 2y_{i+1} - 2m^2 + 1 \\
 &\quad - [2(x_{i+1})^2 + 2y_i^2 - 2y_i - 2m^2 + 1]
 \end{aligned}$$

$$= 4x_i + 2y_{i+1}^2 - 2y_{i+1} - 2y_i^2 + 2y_i + 6$$

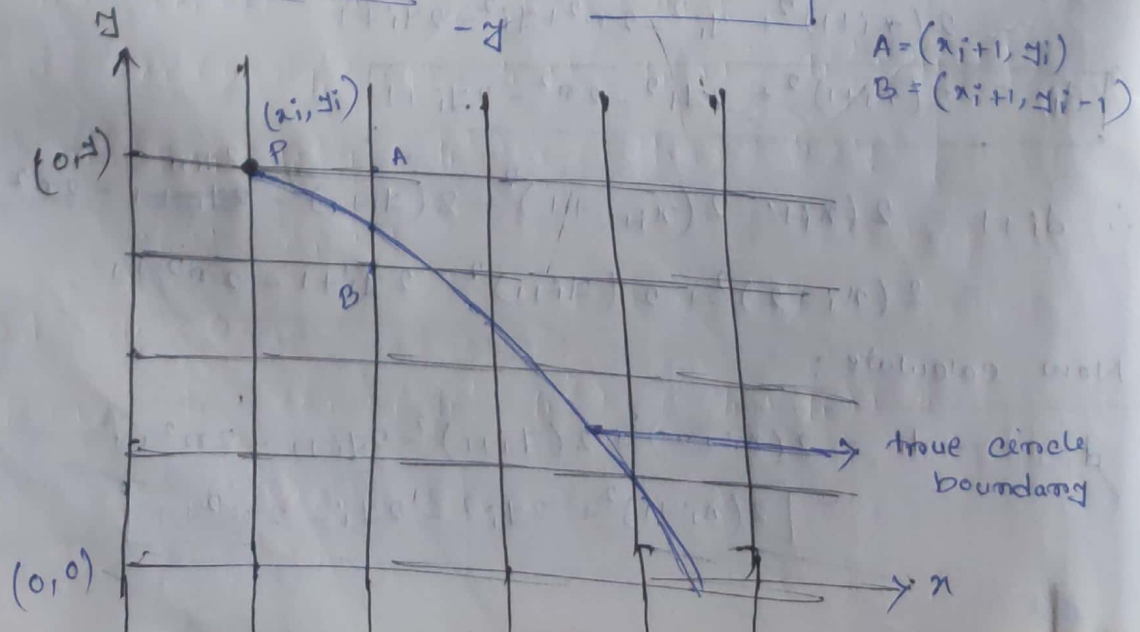
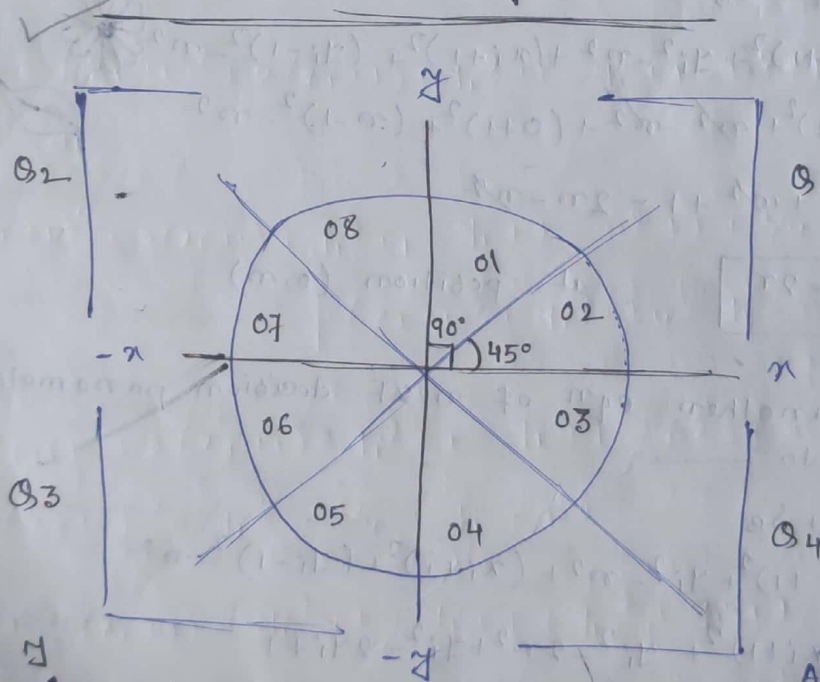
$$d_{i+1} = d_i + 4x_i + 2(y_{i-1})^2 - 2(y_{i-1}) - 2y_i^2 + 2y_i + 6$$

$$d_{i+1} = d_i + 4(x_i - y_i) + 10$$

$$d_{i+1} = d_i + 4x_i + 2x_i^2 - 2x_i - 2x_i^2 + 2x_i + 6$$

$$\therefore d_{i+1} = d_i + 4x_i + 6$$

Circle drawing



Here M is the mid-point of A & B pixel,

$$M = (x_{i+1}, y_i - 1/2)$$

$$= \left(\frac{x_{i+1} + x_{i+1}}{2}, \frac{y_i + y_{i-1}}{2} \right)$$

$$= [(x_{i+1}), (y_i - 1/2)]$$

Circle equation,

$$(x - k)^2 + (y - k)^2 \leq r^2$$

at origin

$$x^2 + y^2 - r^2 = 0$$

$$F_{\text{circle}}(x, y) = x^2 + y^2 - r^2$$

Circle equation at M point,

$$F_{\text{circle}}(x_{i+1}, y_i - 1/2) = (x_{i+1})^2 + (y_i - 1/2)^2 - r^2$$

$$d_i = (x_{i+1})^2 + y_i^2 + 1/4 - y_i - r^2$$

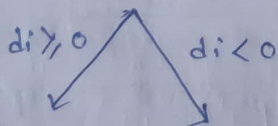
d_i at $(0, r)$

$$= (0+1)^2 + r^2 + 1/4 - r - r^2$$

$$= 1 + 1/4 - r = \boxed{5/4 - r}$$

$= 1.25 - r$

$$d_i = 1.25 - r$$



A is far from the circle, M is outside the circle then B is selected.

$$x_{i+1} = x_i + 1$$

x_{next}

$$y_{i+1} = y_i - 1$$

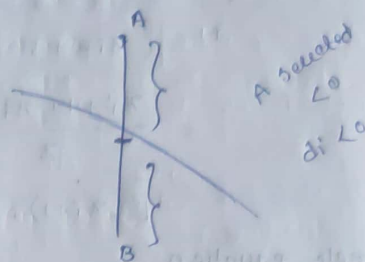
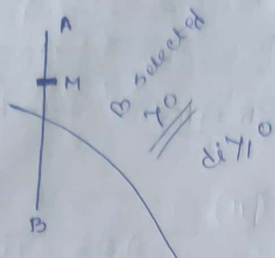
B is far from the circle, M is inside the circle then A is selected then

$$x_{i+1} = x_i + 1$$

x_{next}

$$y_{i+1} \rightarrow y_i$$

y_{next}



← New Next decision Variable →

$$d_{i+1} = \text{Feinole} (x_{i+1} + 1, y_{i+1} - 1/2)$$

$$d_{i+1} = (x_{i+1} + 1)^2 + (y_{i+1} - 1/2)^2 - r^2$$

$$= \left(\frac{(x_i + 1)}{a} + \frac{1}{b} \right)^2 + y_{i+1}^2 + 1/4 - y_{i+1} - r^2$$

$$= (x_i + 1)^2 + 2(x_i + 1) + 1^2 + y_i^2 + 1 + 1/4 - y_{i+1} - r^2$$

Subtraction

$$d_{i+1} - d_i$$

$$(x_i + 1)^2 + 2(x_i + 1) + y_i^2 + 1 - y_{i+1} - r^2 + 1 + 1/4 - [(x_i + 1)^2 + y_i^2 + 1/4 - y_i - r^2]$$

$$= (x_i + 1)^2 + 2(x_i + 1) + y_i^2 + 1 - y_{i+1} - r^2 + 1 + 1/4 - (x_i + 1)^2 - y_i^2 - 1/4 + y_i + r^2$$

$$= d_i + 2(x_i + 1) + y_i^2 + 1 - y_{i+1} - y_i^2 + y_i + 1$$

$$= d_i + 2x_i + 2 + y_i^2 + 1 - y_{i+1} - y_i^2 + y_i + 1$$

$$= \boxed{d_i + 2x_i (y_i^2 + 1 - y_i^2) + (-y_{i+1} + y_i) + 3}$$

$$x_{i+1} = x_i + 1$$

and $d_i < 0$

$$d_{i+1} = d_i + 2x_i + (y_i^2 - y_i^2) + (-y_{i+1} + y_i) + 3$$

$$= \boxed{d_i + 2x_i + 3}$$

$$At \quad d_i \geq 0$$

$$\Rightarrow d_i + 2x_i + (x_i + 1)^2 - y_i^2 + (-x_i + 1 + y_i) + 3$$

$$\Rightarrow d_i + 2x_i + x_i^2 + 1 - 2x_i - x_i^2 + 4$$

$$\Rightarrow d_i + 2(x_i - y_i) + 5$$

$$\boxed{d_i = 1.25 - n}$$

$$\approx 1 - n$$

Q: Circle starting at (0, 5)

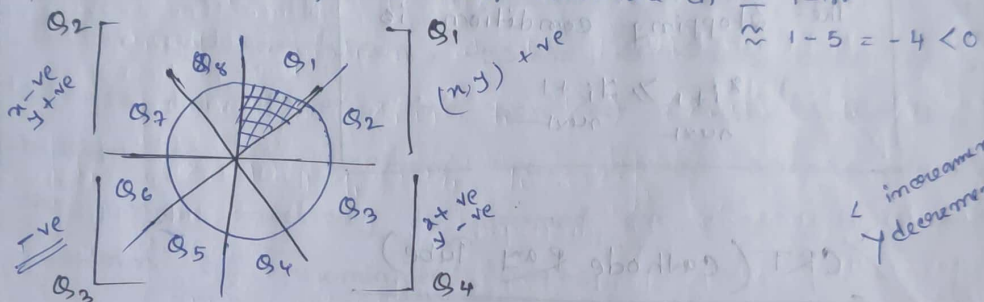
$$x = 0, \quad y = 5$$

$$n = 5$$

$$\boxed{0, n}$$

find point on 1st octant.

$\Rightarrow$ Initial value of decision variable $d_i \approx 1 - n$



	x_i	y_i	x_{i+1}	y_{i+1}	d_{i+1}
	0	5	1	5	-4
	0	5	1	5	-1
	1	5	2	5	4
	2	5	3	4	3
stop *	3	4	4	3	10

$$d_i \geq 0 \quad d_i < 0$$

$$x_{next} = x_i + 1$$

$$y_{next} = y_i - 1$$

$$x_{next} = x_i + 1$$

$$y_{next} = y_i - 1$$

$$d_{i+1} = d_i$$

$$+ 2(x_i - y_i) + 5$$

$$d_{i+1} = d_i + 2x_i + 3$$

$$\textcircled{1} \quad d_{i+1} = -1 < 0$$

$$d_i + 2a_i + b$$

$$= -4 + 2(0) + 3$$

$$= -4 + 0 + 3$$

$$= -1$$

$$\textcircled{2} \quad d_{i+1} = 4 > 0$$

$$= -1 + 2(1) + 3$$

$$= -1 + 2 + 3$$

$$= 4$$

$$\textcircled{3} \quad d_{i+1} = 3 > 0$$

$$= 4 + 2(2-5) + 5$$

$$= 4 + 2(-3) + 5$$

$$= 4 - 6 + 5$$

$$= 4 - 1 = 3$$

$$\textcircled{4} \quad 3 + 2(4-5) + 5$$

$$= 3 + 2 + 5$$

$$= 10$$

The stopping condition is

$$x_{i+1} \geq x_{i+1}$$

next next

✓ CRT (Cathode Ray Tube)

A Beam of electrons (cathode rays) emitted by an electron gun buses through focusing and deflecting systems that direct the beam towards specified positions on the phosphor coated screen. The phosphor then emits a small spot of light at each position contacted by the electron of light at each position from beam. Because the light emitted by the phosphor fades very rapidly, some method is needed for maintaining the screen picture. One way to keep the phosphor glowing is to redraw the picture repeatedly by quickly directing the beam back over the same point. This type of display is called refresh CRT.

The primary components of an electron gun in a CRT are heated metal cathode and control grid, other components are focusing anode, accelerating anode, cathode, phosphor coated screen.

- What are the basic properties unit of a video display unit?

⇒ Resolution and Aspect Ratio.

✓ Raster Scan Display: The most common type of graphics monitor employs a CRT is the Raster Scan Display based on ~~television~~ television technology.

In a Raster Scan system, electron beam is swept across the screen, one row at a time from top to bottom as the electron beam moves across each row, the beam intensity is turned on & off to create a pattern of illuminated spots.

Picture definition is stored in memory area called the Refresh Buffer or Frame Buffer.

These memory holds the set of ~~in~~ Intensity values for the ~~to~~ screen points (dots).

Chapter - 4

Attribute of Output primitives

Any parameter that affects the way a primitive is to be displayed is referred to as an attribute parameter.

Some attribute parameters such as colour, width, size determine the fundamental characteristics of a primitive.

Line Attributes :

Basically straight line segments is defined by its type, width, colour.

Line type: The line type attributes includes —

(i) Solid lines

(ii) Dashed lines

(iii) Dotted lines

(we use line drawing algorithm for solid line representation)

(no. could be displayed by generating an interval)

dash spacing that is equal to the length of the solid section.

A dotted line can be displayed by generating very short dashes with the spacing equal to or greater than the dash size.

Raster line Algorithms display line type attributes by plotting pixels spans. From the various dashed, dotted, and dot-dashed patterns, the line drawing procedure outputs sections of continuous pixels along the line path, skipping over a no. of ~~sk~~ intervening pixels between the solid spans.

(Pixel counts for the span length and inter-span spacing can be specified in a pixel mask), which is a string containing the digits 0 or 1 to indicate which positions to plot along the line path.
 0 — off
 1 — on

For example, the mask 1111000, could be used to display dashed line with a dash length of 4 pixels and inter-dash spacing of 3 pixels.

On a bi-level system, the mask gives the bit values that should be loaded into the frame buffer along the line path to display the selected line type.

• Line width:

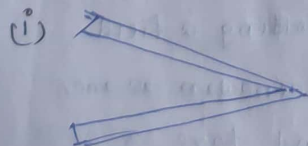
Implementation of line width options depends on the capabilities of the output device. A heavy line on a video monitor could be displayed as a adjacent parallel lines, while a pen plotter might require pen changes.

For master implementation, a standard-width line is generated with single pixels at each sample positions pixel positions as in the Bresenham algorithm.

For lines width slope magnitude greater than 1, we can plot thick lines with horizontal spans, alternating picking a pixel to the right and left of the line path. Although thick lines are generated quickly by plotting horizontal and vertical pixel spans, the displayed width of a line is depended on its slope.

Displaying thick lines using horizontal and vertical pixel spans, leaves pixel gaps at the boundaries between lines of different slopes where there is a shift from horizontal spans to vertical spans.

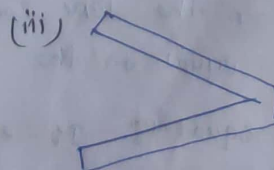
We can generate thick polylines that are smoothly joined at the cost of additional processing at the segment end points. There are three possible methods for smoothly joining two line segments. These are _____



Miter join



Round join



Bevel join

A Miter join is accomplished by extending the outer boundaries of each of the two lines until they meet.

A round join is produced by capping the connection between the two segments with a circular boundary whose diameter is equal to the line width.

A Bevel join is generated by displaying the line segment with butt caps and filling in the triangular cap where the segment meets.

Line Endcaps

The problem with implementing width option using horizontal and vertical pixel spans is that the method produces line whose ends are horizontal or vertical regardless of the slope of the line. This effect is more noticeable with very thick lines. There are three methods we can apply to do so —

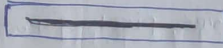
(i) Butt cap



(ii) Round cap



(iii) Projecting square cap



Butt cap: one kind of line cap is the Butt Cap obtained by adjusting the end positions of the component parallel lines so that the thick line is displayed with square ends that are perpendicular to the line path.

Round cap: Round cap obtained by adding a filled semi circle to each butt cap. The circular arcs are centred on the line endpoints and have a diameter equal to the line thickness.

Projecting square cap: Is a third type of line cap. We simply extend the line and add butt caps that are positioned one half of the line width

beyond the specified endpoints.

Line Attribute

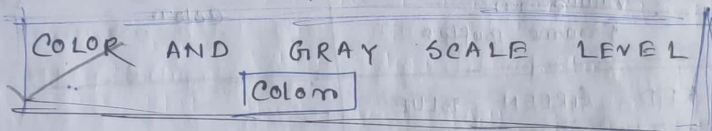
Pen and Brush: • The selected "pen" or "brush" determine the way a line will be drawn.

- pens and brushes have size, shape, color and pattern attribute.

The Curve Attribute

Thickened curves can be produced by —

1. plotting additional pixel
2. Filling the space between two concentric circles.
3. using thickened pen or brush



- Coloms are represented by coloms codes which are positive integers.
- Colom information is stored in frame buffer or in separate table and use pixel values as index to the colom table.
- Two ways to store colom information —
 1. Direct —
 2. Indirect —

16 M x
colom

With the Direct Storage Scheme whenever a particular colom code is specified in an application program, the corresponding binary value is placed in the frame buffer for each component pixel in the output primitive to be displayed in that colom.

A minimum no. of coloms ^{can be} ~~are~~ provided in the scheme with three bits of storage per pixel.

A Miter join

boundaries

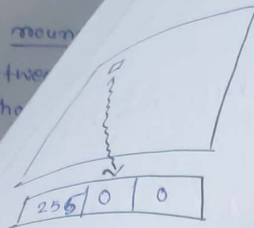
they meet

A miter

between

who

A



Each of the 3 bit position (RGB) is used to control the intensity level of the corresponding electron gun in an RGB monitor.

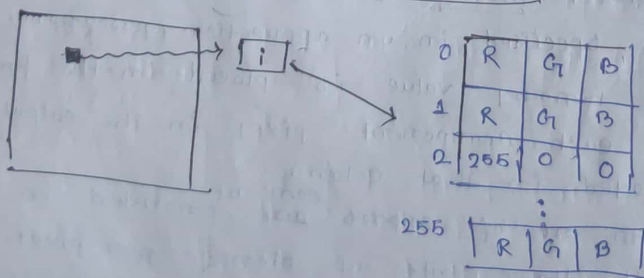
The Eight color codes form a 3-bit per pixel Frame Buffer:

Color	Stored color values in Frame Buffer			Displayed Color
Code	RED	GREEN	BLUE	
0	0	0	0	Black
1	0	0	1	Blue
2	0	1	0	Green
3	0	1	1	Cyan
4	1	0	0	Red
5	1	0	1	Magenta
6	1	1	0	Yellow
7	1	1	1	White

0 → Black
1 → White

Diagram:

Color maps (LUT's)



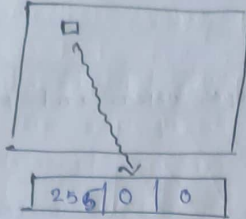
Store RGB in Array form

Solid:

Pattern

Direct

Full-color (RGB) displays



Each of the 8 bit position (RGB) is used to control the intensity level of the corresponding electron gun in an RGB monitor.

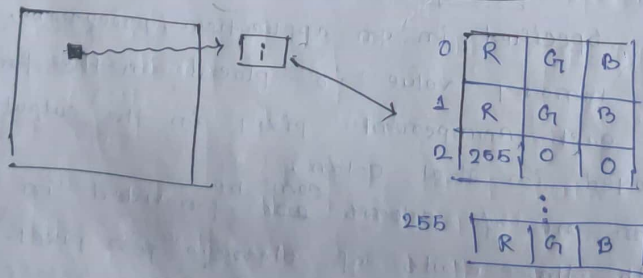
The Eight color codes form a 3-bit per pixel frame buffer:

Color	Stored color values in frame buffer			Displayed color
Code	RED	GREEN	BLUE	
0	0	0	0	Black
1	0	0	1	Blue
2	0	1	0	Green
3	0	1	1	Cyan
4	1	0	0	Red
5	1	0	1	Magenta
6	1	1	0	Yellow
7	1	1	1	White

0 → Black
1 → white

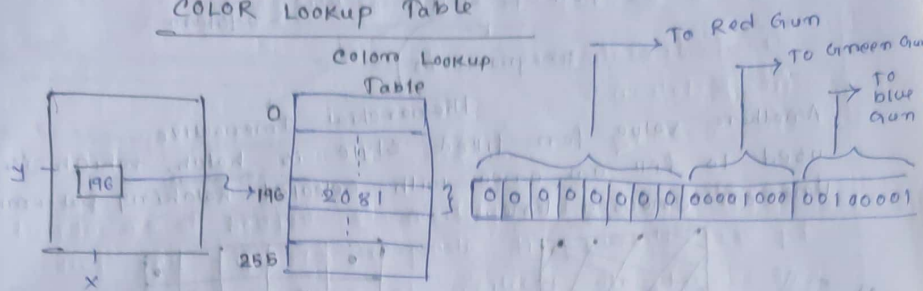
Indirect:

Color maps (LUT's)



Store RGB in Array form

COLOR Lookup Table



A color Lookup table with 24 bits per entry accessed from a frame buffer with 8 bits per pixel. A value of 196 stored at pixel position (x, y) references the location in this table containing the value 2081. Each 8-bit segment of this entry controls the intensity level of one of the three electron guns in an RGB monitor.

Area filled Attribute

There are 3 methods for area filled attribute —

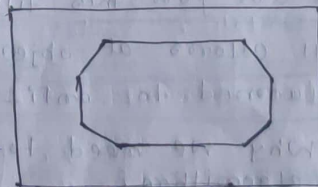
- (i) Hollow (ii) Solid (iii) pattern

Hollow: There is a boundary with different color, interior color is same with background.

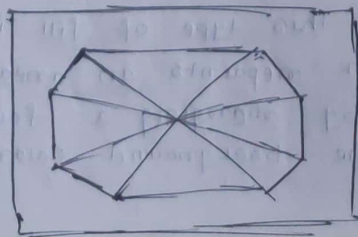
Area filled Attribute

Solid: Filled with a solid color. color up to and including the borders.

Image or object filled with solid color.



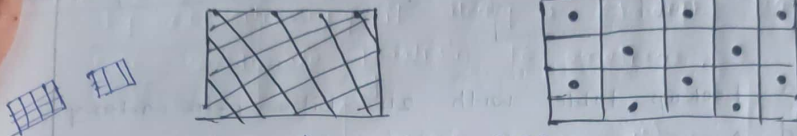
Pattern: Pattern control by other table. Pattern is the object filled with specific color. we can choose particular pattern area for filling the pattern with color.



This is the pin — PH I Graphics —> System
 programmers

Hierarchical Interactive

Another value for filled style is hatch, which is used to fill an area with selected hatching pattern-pattern line on crossed line.



Other fill options are —

Color-Blended Fill Region
 edge type, edge width and edge color of a region.

Pattern fill :

(The process of filling an area with a rectangular pattern is called tiling). ~~An rectangular pattern~~.
 and Rectangular filled pattern are sometimes referred to as pattern filling.

Soft fill : Modified boundary fill and flood fill

procedures that are applied to repainted areas so that the fill color is combined with the boundary color, background colors are referred to as soft fill or tint-fill algorithm.

one use for this fill methods is to soften the fill colors at object borders that have been blurred to anti-alias the edges.



Q: why we need to use the soft fill ^{on} and ¹ tint-fill algorithm.

Another is to allow repainting of a colour area that was ~~at~~ originally filled with a semi-transparent brush, where the current colour is then a mixture of the brush colors and the background colors.

An example of this type of fill is — linear soft fill. Algorithm repaints an area that was originally painted by merging a foreground color with a single background color.

Chapter - 5

Basic Transformations

- (i) Translation
- (ii) Rotation
- (iii) Scaling

Translation: A translation is applied to an object by repositioning it along a straight line path from one coordinate location to another.

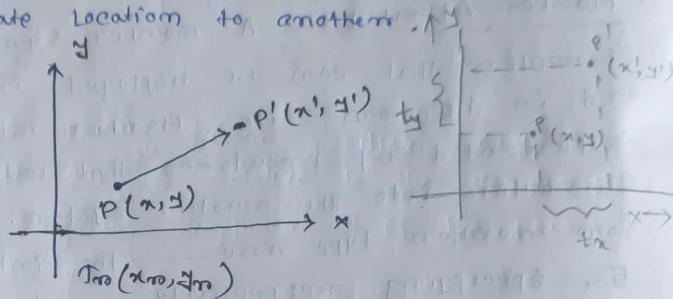


Fig: Translating a point from position P to position P' .

We translate a 2D point by adding a

translation distances t_x and t_y to the original coordinate position x, y to move the point to a new location $P'(x', y')$.

$$x' = x + t_x$$

$$y' = y + t_y$$

The translation distance pair $(t_x \text{ and } t_y)$ is called translation vector or shift vector.

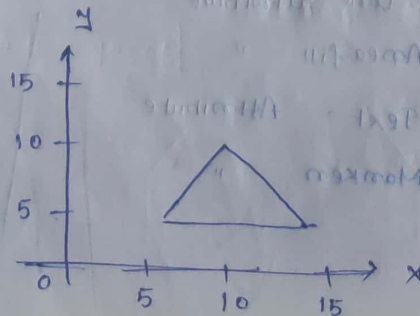
Matrix form:

$$P = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \text{ shift } P' = \begin{bmatrix} x'_1 \\ x'_2 \end{bmatrix}$$

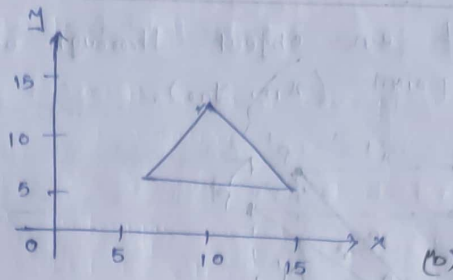
$$T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

$$\therefore P' = P + T$$

Polygon:



(a)



Moving a polygon from position (a) to position (b) with the translation vector $(2, 2) [(+x, +y)]$

Ex: Polygon

$$A = (2, 5)$$

$$B = (9, 10)$$

$$C = (10, 2)$$

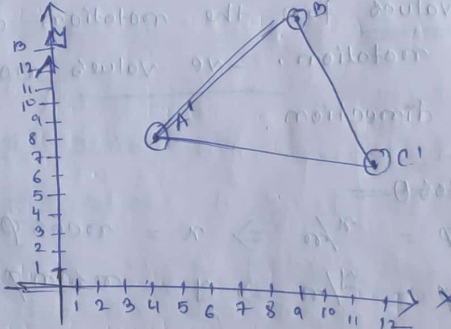
$$(+x, +y) = (2, 2)$$

$$\Rightarrow A' = (4, 8)$$

$$B' = (11, 12)$$

$$C' = (12, 4)$$

$$[x', y'] = [x, y] + [+x, +y]$$



Rotation:

A 2D rotation is applied to an object by re-positioning it along a circular path in the (x, y) plane. To generate a rotation we need to specify rotation angle θ at position (x_m, y_m) of the rotation point also called pivot point about which the object is to be rotated.

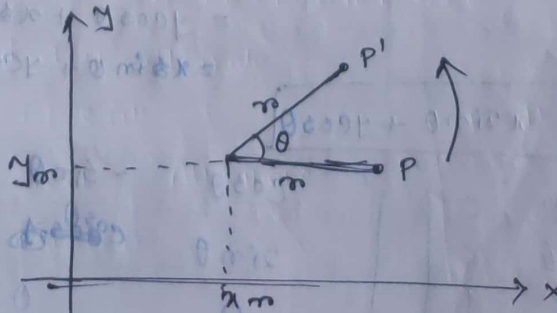
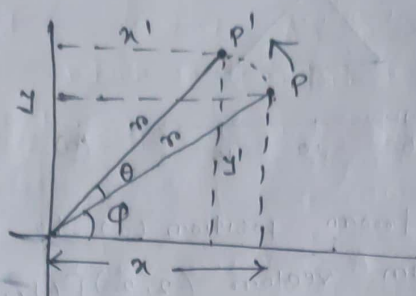


Fig - 1

Rotation of an object through angle θ about the pivot point (x_m, y_m) .

through origin



Rotation of a point from position (x, y) to position (x', y') through an angle θ relative to the coordinate origin. The original angular displacement of the point from the x -axis is ϕ .

N.B: +ve values for the rotation angle, define ~~counter~~ anti-clockwise rotation, -ve values rotate objects in the clockwise direction.

~~cos theta~~

$$\left. \begin{aligned} \cos \phi &= x/r \Rightarrow x = r \cos \phi \\ \sin \phi &= y/r \Rightarrow y = r \sin \phi \end{aligned} \right\} \text{--- (i)}$$

$$\begin{aligned} \cos(\theta + \phi) &= x'/r \Rightarrow x' = r \cos(\theta + \phi) = r \cos \theta \cos \phi - r \sin \theta \sin \phi \\ \sin(\theta + \phi) &= y'/r \Rightarrow y' = r \sin(\theta + \phi) = r \cos \theta \sin \phi + r \sin \theta \cos \phi \end{aligned} \quad \text{--- (ii)}$$

$$\therefore \boxed{x' = x \cos \theta - y \sin \theta}$$

$$\begin{aligned} \text{Again, } \sin(\theta + \phi) &= y'/r \Rightarrow y' = r \sin(\theta + \phi) \\ &= r \cos \theta \sin \phi + r \sin \theta \cos \phi \\ &= y \cos \theta + x \sin \theta \\ &= x \sin \theta + y \cos \theta \end{aligned}$$

$$\therefore \boxed{y' = x \sin \theta + y \cos \theta}$$

$$\begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Ex: Original position $P(4, 3)$, Angle $\theta = 45^\circ$.

$$\begin{aligned}\Rightarrow x' &= x \cos \theta - y \sin \theta \\ &= 4 \cos 45^\circ - 3 \sin 45^\circ \\ &= 4 \times \frac{1}{\sqrt{2}} - 3 \times \frac{1}{\sqrt{2}} \\ &= \frac{1}{\sqrt{2}}\end{aligned}$$

$$\begin{aligned}y' &= x \sin \theta + y \cos \theta \\ &= 4 \sin 45^\circ + 3 \cos 45^\circ \\ &= 4 \times \frac{1}{\sqrt{2}} + 3 \times \frac{1}{\sqrt{2}} \\ &= \frac{7}{\sqrt{2}}\end{aligned}$$

$\therefore P' = \left(\frac{1}{\sqrt{2}}, \frac{7}{\sqrt{2}}\right)$ at angle 45°

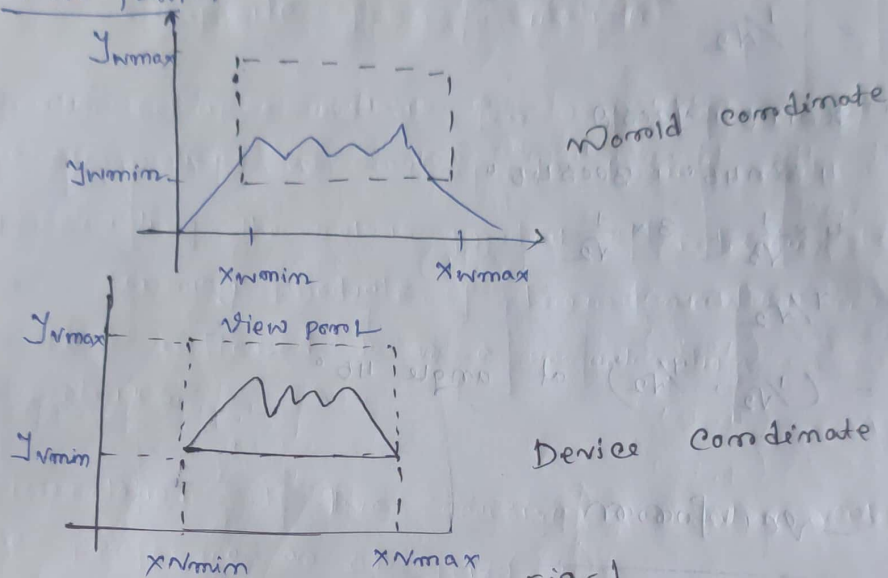
In -ve form

$$\begin{aligned}x' &= x \cos(-\phi) - y \sin(-\phi) \\ &= x \cos \phi + y \sin \phi\end{aligned}$$
$$\begin{aligned}y' &= x \sin(-\phi) + y \cos(-\phi) \\ &= -x \sin \phi + y \cos \phi\end{aligned}$$

Chapter - 6

2-D viewing

A world coordinate area selected for display is called a window. An area on a display device to which a window is mapped is called view port.



The window defines what is to be viewed, the view port defines where it is to be displayed.

2-D viewing is the mechanism for displaying views of an object on an output device.

The mapping of a part of a world coordinate scene to device coordinates is referred to as a viewing transformation.

- (i) When we are interested to display certain portion of the drawing, enlarge the portion, windowing technique is used.
- (ii) Technique for not showing the part of the drawing which one is not interested is called clipping.
- (iii) In 2-D viewing transformation is simply referred as the window-to-view port transformation or the windowing transformation.

(iv) Mapping a window onto a view port involves converting from one coordinate system to another.

(v) If the window and view port are in standard position —

(a) It involves translation and scaling.

(b) If the window and/or view port are not in standard position, then extra transformation which is rotation is required.

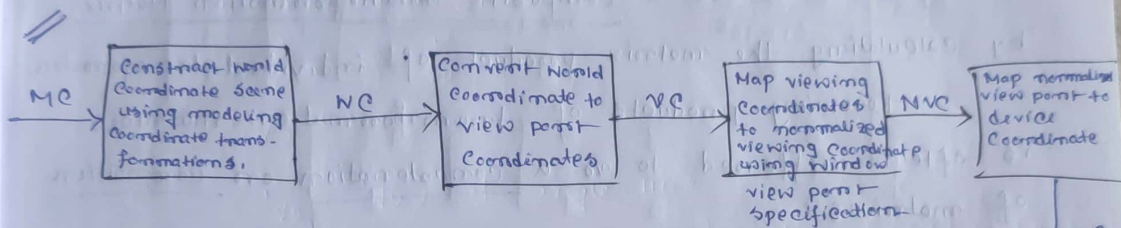


Fig-2 : The 2-D viewing transformation pipeline

Some graphics packages that provide window and view port operations allow only standard rectangles but a more general approach is to allow the rectangular window to have any orientation.

We carry out the viewing transformation in several steps as indicated in the above fig.

Chapter-5.

Scaling :

A scaling transformation alter the size of an object. This operation can be carried out for polygon by multiplying the coordinate's value (x, y) of each vertex by scaling coordinates (x', y') .

If δx and δy are > 1 then zooming.
If δx and δy are < 1 then zoom out.
If δx and δy are $= 1$ then No changes.

$$\left. \begin{aligned} x' &= x \cdot \delta x \\ y' &= y \cdot \delta y \end{aligned} \right\} \text{--- (1)}$$

Scaling factor δx scales objects in the x direction, while δy scales in the y direction. The above eqn (1) can also be written in the matrix form :

$$\begin{matrix} P' & & S & & P \\ \begin{bmatrix} x' \\ y' \end{bmatrix} & = & \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} & \cdot & \begin{bmatrix} x \\ y \end{bmatrix} \end{matrix}$$

$$\text{or, } \boxed{P' = S \cdot P}$$

Composite Transformation

We can set up a matrix for any sequence of transformations as a Composite transformation Matrix, by calculating the matrix product of individual transformations. Forming products of transformation matrices is often referred to as a concatenation or composition of matrices.

① Translation : If two successive translations, ~~vectors~~ are performed on a point P :

$$P' = \begin{bmatrix} 1 & 0 & tx_1 + tx_2 \\ 0 & 1 & ty_1 + ty_2 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Two successive translations, always additive

$$T(tx_1, ty_1)$$

$$T(tx_2, ty_2)$$

$$\begin{aligned} P' &= \{ P \cdot T(tx_1, ty_1) \} \cdot T(tx_2, ty_2) \\ &= P \{ T(tx_1, ty_1) \cdot T(tx_2, ty_2) \} \end{aligned}$$

In matrix format,

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ tx_1 & ty_1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ tx_2 & ty_2 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{aligned} T &\rightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ tx_1 + tx_2 & ty_1 + ty_2 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \end{aligned}$$

Q: obtain the final coordinates after two successive translations on point $P = (2, 3)$ with translation vectors $(4, 3)$ & $(-1, 2)$.

$$\Rightarrow P' = \begin{bmatrix} 1 & 0 & 4-1 \\ 0 & 1 & 3+2 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 5 \\ 8 \\ 1 \end{bmatrix}$$

$\therefore$ final coordinates after translation are $(5, 8)$.

② Rotation : Two successive rotations are performed

$$\text{as } P' = P \{ R(\theta_1) \cdot R(\theta_2) \}$$

$$R(\theta_1) \cdot R(\theta_2) = \begin{bmatrix} \cos \theta_1 & \sin \theta_1 & 0 \\ -\sin \theta_1 & \cos \theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta_2 & \sin \theta_2 & 0 \\ -\sin \theta_2 & \cos \theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \theta_1 \cos \theta_2 - \sin \theta_1 \sin \theta_2 & \cos \theta_1 \sin \theta_2 + \sin \theta_1 \cos \theta_2 & 0 \\ -\sin \theta_1 \cos \theta_2 - \sin \theta_2 \cos \theta_1 & -\sin \theta_1 \sin \theta_2 + \cos \theta_1 \cos \theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos(\theta_1 + \theta_2) & \sin(\theta_1 + \theta_2) & 0 \\ -\sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 0 \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Q1. Obtain the final coordinates after two rotations on point $P = (6, 9)$ with rotation angles are 90° and 60° respectively.

$$\begin{aligned} \Rightarrow P' &= \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 0 \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 6 \\ 9 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 6 \\ 9 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 6 \\ 9 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} -9 \\ 6 \\ 1 \end{bmatrix} \end{aligned}$$

$\therefore$ The final coordinates after rotation are $(-9, 6)$

Two Successive Scaling : (multiplicative)

$$P' = P \cdot \left\{ s(sx_1, sy_1), s(sx_2, sy_2) \right\}$$

$$= \begin{bmatrix} sx_1 & 0 & 0 \\ 0 & sy_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} sx_2 & 0 & 0 \\ 0 & sy_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} sx_1 sx_2 & 0 & 0 \\ 0 & sy_1 sy_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$\therefore$ Two successive scaling are performed as

$$P' = \begin{bmatrix} sx_1 sx_2 & 0 & 0 \\ 0 & sy_1 sy_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot P$$

This concept can be extended for any no. of successive scaling.

Question Obtain the final coordinates after two scaling on line $PQ = [P(2,2), Q(8,8)]$ with scaling factors s_1 are $(2,2)$ and $(3,3)$ respectively.



$$s_1 = (2,2) \quad s_2 = (3,3)$$

$$s_{x1} = 2$$

$$s_{x2} = 3$$

$$s_{y1} = 2$$

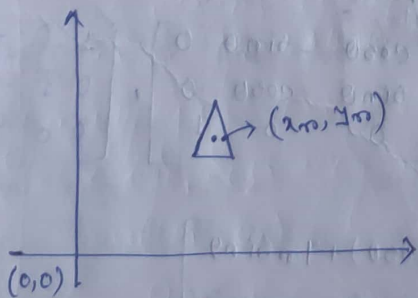
$$s_{y2} = 3$$

$$PQ = \begin{bmatrix} 2 & 8 \\ 2 & 8 \\ 1 & 1 \end{bmatrix}$$

$$P' = \begin{bmatrix} 6 & 0 & 0 \\ 0 & 6 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 & 8 \\ 2 & 8 \\ 1 & 1 \end{bmatrix}$$

$$Z = \begin{bmatrix} 12 & 48 \\ 12 & 48 \\ 1 & 1 \end{bmatrix}$$

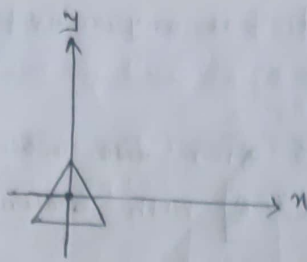
Rotation with respect to pivot point :



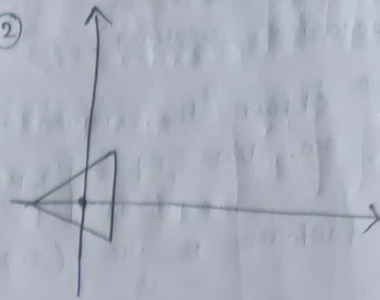
(a) Original position of object & pivot point.

STEPS :

①



②

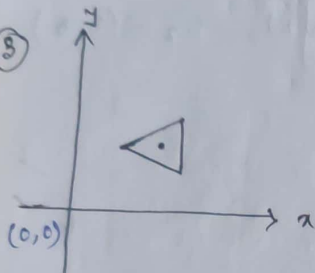


(b) Translation of object so that pivot point $P_t(x_m, y_m)$ is at origin.

(c) $R(\theta)$

Rotation about origin

③



(d) $T^{-1} \rightarrow$ Translation of object so that the pivot point is returned to position (x_m, y_m) .

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$\therefore T^{-1} = \begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix}$$

From the three steps we get,

Translation:
respect to pivot
point

$$= \begin{bmatrix} 1 & 0 & x_m \\ 0 & 1 & y_m \\ 0 & 0 & 1 \end{bmatrix}$$

Rotation with angle θ

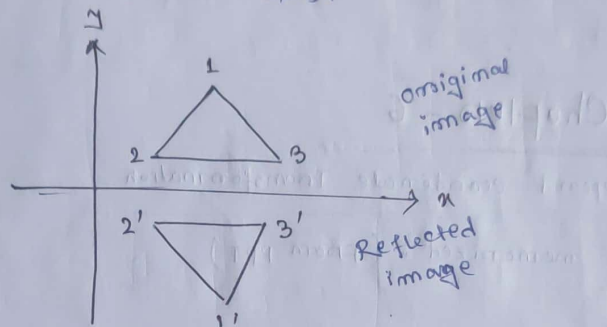
$$= \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_m \\ 0 & 1 & -y_m \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \theta & -\sin \theta & x_m(1 - \cos \theta) + y_m \sin \theta \\ \sin \theta & \cos \theta & y_m(1 - \cos \theta) - x_m \sin \theta \\ 0 & 0 & 1 \end{bmatrix}$$

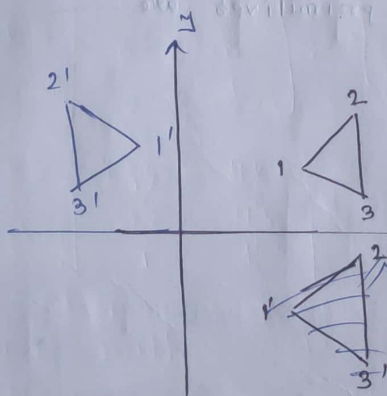
=

Other Transformation :

Reflection : A reflection is a transformation that produces a mirror image of an object. The mirror image from a 2D reflection is generated relative to an axis of reflection by rotating the object 180° about the reflection axis.



① Reflection of an image about x-axis.

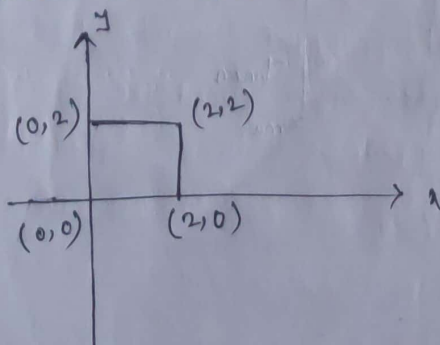


② Reflection of an image about y-axis

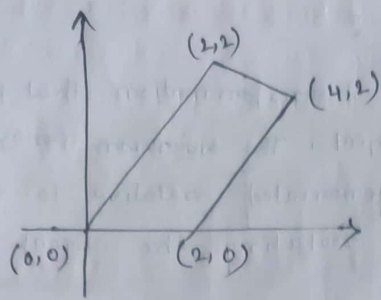
change the shape

Shear : A transformation that distorts the shape of an object such that the transformed shape appears as if the object were composed of internal layers that had been caused to slide over each other is called a shear.

Ex:



let us assume shear factor $Sh_x = 2$



Shearing

→ In x-axis:

$$x' = x + sh_x \cdot y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & sh_x \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

→ In y-axis:

$$y' = y + sh_y \cdot x$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ sh_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Chapter - 6

Window-to-viewpoint coordinate transformation

2x & 3y have to memorized → (From PPT)

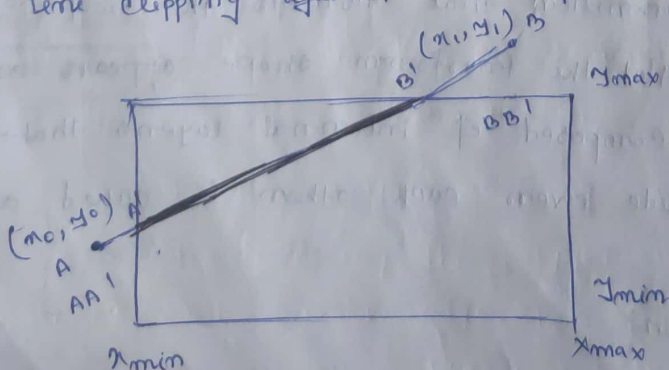
~~Some~~

Several algorithm for clipping primitives are —

- ✓ (i) Point clipping
- (ii) Line "
- (iii) Area / polygon clipping
- (iv) Curve clipping
- (v) Text "

Line clipping

Line clipping against rectangles.



problem: Given

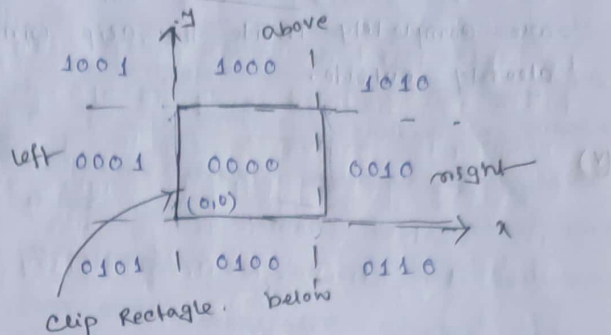
$$\begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \end{bmatrix}$$

The Cohen-Sutherland Line-Clipping Algorithm

Region and outcodes

0 0 0 0
left right above below



Place the value of 1st bit is '1' if it is in left side.

IBRL

① Put the 1st bit '1' that means the image is in the left side of viewport.

② 3rd bit '1' below.

0101

③ 4th bit '1' above.

↪ left and below.

④ 2nd bit '1' right.

(i) C-S is one of the oldest and popular line-clipping algo.

(ii) Every line end point in a picture is assigned a four digit binary code, called region code, that identifies the location of the point relative to the boundaries of the clipping rectangle.

(iii) Bit values in the region code are determined by comparing the end point coordinate values (x, y) to the clip boundaries.

bit '1' is set to 1 if $x < x_{min}$. The other three

bits can be determined using similar comparisons.

The steps are —

(i) calculate differences between end point coordinates and clipping boundaries.

(ii) use the resultant sign bit of each difference calculations to set the corresponding values in the region code.

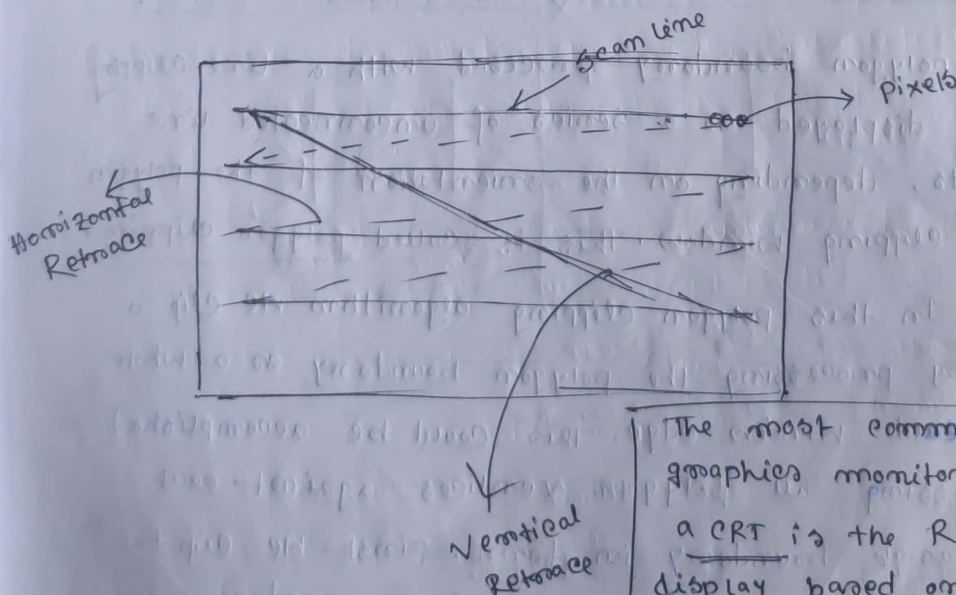
Raster Scan Display

A Raster Scan Display is based on intensity control of pixels in the form of a rectangular box called master on the screen. Information of ON and OFF pixels is stored in refresh buffer or frame buffer. Televisions in our house are based on Raster Scan Method. The raster scan system can store information of each pixel position, so it is suitable for each realistic display of objects. Raster scan provides a refresh rate of 60 to 30 frames per second.

In a Raster Scan System electron beam is swept across the screen. Beam refreshing is two types —

- (i) Horizontal Retracing
- (ii) Vertical Retracing

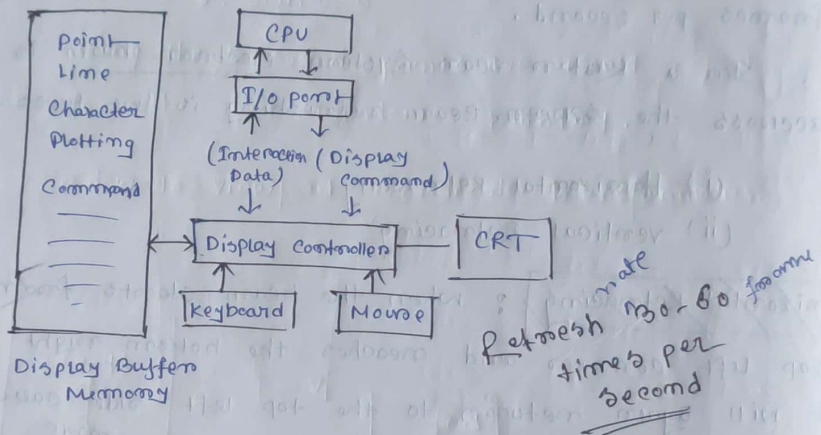
Horizontal Retracing : when the beam starts from the top left corner and reaches the bottom right side, it will again return to the top left side called as vertical retrace. Then it will again ~~move~~ ^{move} horizontally from top to bottom called as horizontal retrace.



The most common type of graphics monitor employed in a CRT is the Raster Scan display based on television technology.

Random Scan Display

In the Random Scan (vector scan) technique, the electron beam is directed only to the part of the screen where the pictures is to be drawn rather than scanning from left to right and top to bottom as in raster scan. It is also called as vector display, stroke writing display or calligraphic display.



The Sutherland-Hodgeman Polygon Clipping Algo

A polygon boundary processed with a line clipping may be displayed as a series of unconnected line segments, depending on the orientation of the polygon to the clipping window. This is called polygon clipping.

In this polygon clipping algorithm we clip a polygon by processing the polygon boundary as a whole against each window edge. This could be accomplished by processing all polygon vertices against each clip rectangle boundary in turn. First we clip the polygon against the left boundary to produce new sequence of vertices. The new set of vertices could

then be successively passed to a right boundary clipper, bottom boundary clipper and top boundary clipper.

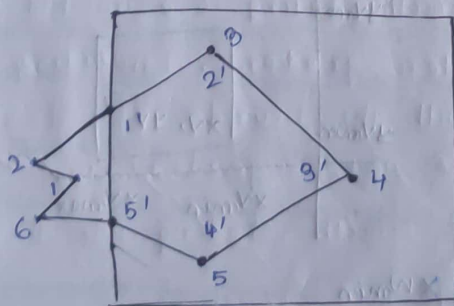
There are 4 possible codes when processing vertices in sequence around the perimeter of a polygon.

(i) If the 1st vertex is outside the window boundary and 2nd vertex is inside the boundary, both the intersection point of the polygon edge with the window boundary and the second vertex are added to the output vertex list.

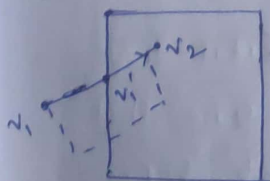
(ii) If both input vertices are inside the window boundary, only the second vertex is added to the output vertex list.

(iii) If the first vertex is inside the window boundary, and the second vertex is outside, only the edge intersection with the window boundary is added to output vertex list.

(iv) If both input vertices are outside the window boundary, nothing is added to the output vertex list.

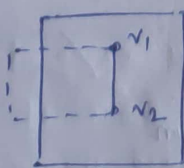


Case-I



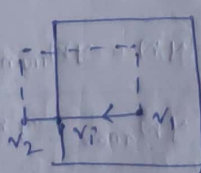
out $\rightarrow$ in
save $\rightarrow v_1', v_2$

Case-II



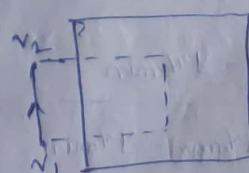
in $\rightarrow$ in
save $\rightarrow v_2$

Case-III



in $\rightarrow$ out
save $\rightarrow v_1'$

Case-IV



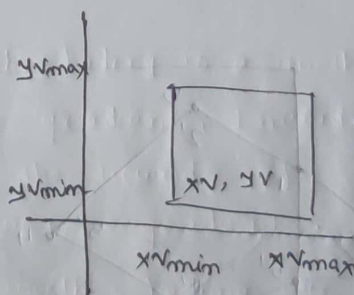
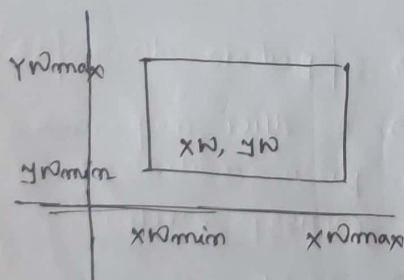
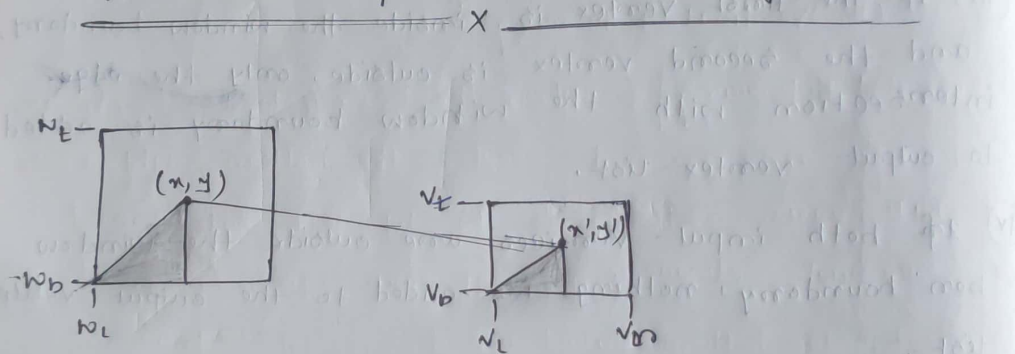
out $\rightarrow$ out
save $\rightarrow$ nothing

The vertices 1 and 2 are the outside of the window. Moving along to vertex 3 which is inside, we calculate the intersection and save both the intersection point (1') and vertex 3. Vertices 3 and 4 both are inside and also 4 and 5 both are inside then save 4 and 5. Then 5 is inside and 6 is outside then save 5' (intersection point)

output list $\rightarrow 1', 3, 4, 5, 5'$

using the 5 saved points we would repeat the process for the next window boundary.

Window-To-viewport Coordinate Transformation



$$\frac{x' - x'_{\min}}{x'_{\max} - x'_{\min}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

$$\frac{y' - y'_{\min}}{y'_{\max} - y'_{\min}} = \frac{y - y_{\min}}{y_{\max} - y_{\min}}$$

From these two equations we derived,

$$x_v = x_{vmin} + (x_w - x_{wmin}) \delta x$$

$$y_v = y_{vmin} + (y_w - y_{wmin}) \delta y$$

Where the scaling factors are

$$\delta x = \frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}}$$

$$\delta y = \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}}$$

The sequence of transformations are

1. perform a scaling transformation using a fixed-point position of (x_{wmin}, y_{wmin}) that scales the window area to the size of the viewport.
 2. Translate the scaled window area to the position of the viewport.
- Relative proportions of objects are maintained if the scaling factors are the same ($\delta x = \delta y$). Otherwise, window objects will be stretched or contracted in either x or y direction when displayed on output device.

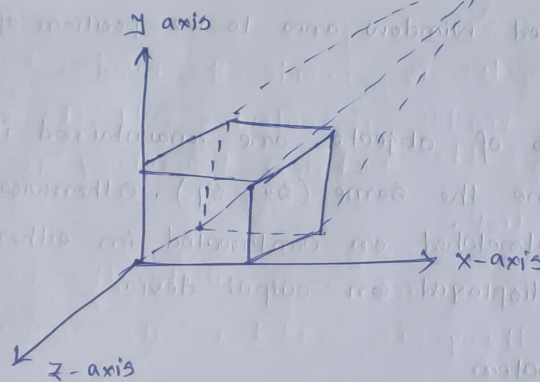
~~How about character~~

Q: How about character strings when map to viewport?

- Maintains a constant character size (apply when standard character fonts cannot be changed).
- If character size can be changed, then windowed will be applied like other primitives.
- For characters formed with curve segments, the mapping to viewport is carried through sequence of line transformation.

① Fractals : Fractals are very complex pictures generated by a computer from a single formula. They are ~~consider~~ created using iterations. This means one formula is repeated with slightly different values over and over again, ~~ke~~ taking into account the results from the previous iteration. Fractals are used in many areas such as — Astronomy, Biology / Chemistry, others etc.

② Vanishing point : A vanishing point is a point on the image plane of a perspective rendering where the 2D perspective projections of mutually parallel lines in 3-D space appear to converge.



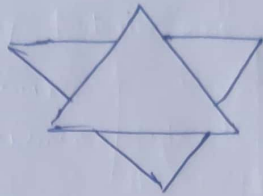
③ Smooth shading : Smooth shading is a method of shading a region with a color gradient. Including color information along with the bounds of drawing primitive, specifies the color gradient.

Two types of smooth shading are there, —

- (i) Gouraud shading
- (ii) Phong shading.

④ Koch curve : The Koch curve is a fractal that starts with a simple pattern made of a line that is divided into 3 equal parts. Erase the middle segment and replace it with an upsidedown "v" shape, and

now the whole pattern is made up of four line segments. The total length of the line is evidently $\frac{4}{3}$.



Surfaces : In technical applications of 3D computer graphics such as computer-aided design and computer-aided manufacturing, surfaces are one way of representing objects.

Shadow : Shadows are created by testing whether a pixel is visible from the light source, by comparing the pixel to a z-buffer or depth image of the light source's view, stored in the form of a texture.